



UNIVERSIDADE FEDERAL DE SÃO PAULO
INSTITUTO DE CIÊNCIA E TECNOLOGIA

**Uso de Inteligência Artificial no Desenvolvimento
do Aplicativo MaMobi - Gestão de Eventos**

Aryssa Shikibu
Bruno de Abreu
Sasha Rodela Steidle

São José dos Campos - SP
2026

Sumário

Introdução.....	3
1.1 O problema que o controle de versão resolve.....	3
1.2 Git: a ferramenta padrão da indústria.....	3
Exemplo do projeto: branches do Jira.....	4
1.3 GitHub e GitLab: onde o repositório vive.....	4
Pull Request: a revisão de código.....	5
1.4 .gitignore e o que NÃO vai para o repositório.....	5
Métodos Ágeis.....	5
2.1 Por que métodos ágeis?.....	5
O Manifesto Ágil em quatro valores.....	5
2.2 Scrum: o framework mais usado.....	6
Os três papéis do Scrum.....	6
Os artefatos do Scrum.....	6
Os eventos do Scrum.....	6
2.3 O Backlog em detalhe: Épicas, Stories e Tasks.....	7
Épicas (EP).....	7
Stories (ST).....	7
Tasks (TK) e Sub-tasks (ST-K).....	7
2.4 Kanban: visualizando o fluxo de trabalho.....	7
WIP Limit: evitar sobrecarga.....	8
2.5 Estimativas: Planning Poker e Story Points.....	8
Arquitetura de Software.....	8
3.1 O que é arquitetura de software?.....	8
3.2 Separação de responsabilidades: o princípio central.....	9
3.3 Padrões arquiteturais: MVC e Arquitetura em Camadas.....	9
MVC (Model-View-Controller).....	9
Arquitetura em Camadas.....	10
DevOps.....	10
4.1 O que é DevOps?.....	10
4.2 CI/CD: o coração do DevOps.....	10
Integração Contínua (CI).....	10
Entrega Contínua (CD).....	11
4.3 Ambientes: desenvolvimento, staging e produção.....	11
4.4 Variáveis de ambiente.....	12
4.5 TDD: testes antes do código.....	12
Como Tudo se Conecta.....	12
5.1 A jornada de uma feature do Jira até a produção.....	12
Inteligência Artificial como Ferramenta de Desenvolvimento.....	13
6.1 Por que falar de IA neste guia?.....	13
6.2 O que a IA faz bem no desenvolvimento de software.....	13
Explicar código desconhecido.....	13
Gerar código de partida (boilerplate).....	14

Depurar erros (debugging).....	14
Escrever testes.....	14
Entender conceitos novos rapidamente.....	14
Revisar Pull Requests.....	15
6.3 Como pedir melhor: a arte do prompt.....	15
6.4 Onde tomar cuidado.....	15
A IA erra - e com confiança.....	16
Não substitui entender o problema.....	16
Cuidado com informações sensíveis.....	16
Versões de bibliotecas e frameworks.....	16
Front-End.....	17
HTML.....	17
CSS.....	20
JavaScript.....	22
Back-End.....	23
Passo a passo.....	23
1. Escolha seus aliados.....	23
2. Saber exatamente o que quer fazer.....	23
3. Esqueleto do projeto.....	23
4. Ciclo de construção e aprimoramento.....	24
Recomendações chave.....	25
Defina Restrições de Design Imediatamente.....	25
Forneça Contexto Estruturado.....	25
Delegue a Lógica, mas Exija a Validação.....	25
Force o uso de Idiomatismos da Biblioteca.....	25

Introdução

Antes de começar a escrever código, todo time de desenvolvimento precisa dominar um conjunto de práticas, ferramentas e conceitos que tornam o trabalho colaborativo possível e sustentável. Este guia apresenta esses fundamentos de forma prática, usando como fio condutor um projeto desenvolvido durante a disciplina Práticas em Projetos Extensionistas: a Plataforma Digital de Gestão de Eventos inspirado nas demandas das Mães Mobilizadoras.

Ao longo do texto você vai entender por que escolhemos certas ferramentas, como o time se organiza para entregar software sem caos, e como a arquitetura de um sistema real é planejada antes de qualquer linha de código ser escrita.

Repositórios e Controle de Versão

1.1 O problema que o controle de versão resolve

Imagine que três pessoas do time estão trabalhando ao mesmo tempo: uma está criando a tela de cadastro de evento, outra está ajustando o banco de dados e a terceira está corrigindo um bug na listagem. Sem nenhuma ferramenta de controle, como cada uma salva o próprio trabalho sem apagar o da colega? E se uma mudança quebrar tudo - como voltar ao estado anterior?

Controle de versão é o conjunto de práticas e ferramentas que responde a essas perguntas. Ele registra cada modificação feita no código ao longo do tempo, identifica quem fez o que e quando, e permite desfazer alterações de forma segura.

1.2 Git: a ferramenta padrão da indústria

Git é o sistema de controle de versão mais usado no mundo. Ele opera de forma distribuída: cada pessoa do time possui uma cópia completa do histórico do projeto no próprio computador e pode trabalhar sem internet. As mudanças são sincronizadas quando necessário.

Termo	O que significa na prática
Repositório (repo)	A pasta do projeto monitorada pelo Git. Contém o código e todo o histórico de alterações.
Commit	Um 'ponto de salvamento' do código. Cada commit tem uma mensagem descrevendo o que mudou.
Branch	Uma linha paralela de desenvolvimento. Permite trabalhar em uma funcionalidade sem afetar o código principal.
Merge	Unir o trabalho de uma branch de volta ao código principal após revisão.
Clone	Baixar uma cópia do repositório para o próprio computador.

Push / Pull

Push: enviar suas mudanças para o servidor. Pull: baixar as mudanças dos colegas.

Exemplo do projeto: branches do Jira

O Jira é uma ferramenta de gerenciamento de projetos amplamente usada em times de desenvolvimento de software. Ele permite criar, organizar e rastrear tarefas ao longo do ciclo de desenvolvimento — desde a ideia inicial até a entrega. No Jira, cada unidade de trabalho recebe um identificador único (como TK-01 ou ST-03), um responsável, uma descrição detalhada e um status que avança conforme o trabalho progride (A Fazer, Em Progresso, Em Revisão, Concluído). Isso torna o progresso do time visível para todos e facilita a priorização do que deve ser feito a cada sprint.

No projeto MaMobi, cada tarefa do Jira (suponha TK-01, TK-02, ST-01...) virou uma branch separada no repositório. Isso significa que o trabalho de criar o formulário de cadastro de evento (TK-01) nunca entrou em conflito com o trabalho de criar o endpoint de API (TK-02), mesmo sendo desenvolvidos ao mesmo tempo.

```
# Exemplo de fluxo de trabalho no projeto

# 1. Baixar a versão mais recente do código
git pull origin main

# 2. Criar uma branch para a tarefa TK-01 (formulário de cadastro)
git checkout -b feature/TK-01-formulario-cadastro

# 3. Trabalhar no código...

# 4. Salvar o progresso com um commit descritivo
git add .
git commit -m "TK-01: adiciona formulário de cadastro com validação
mobile-first"

# 5. Enviar para o servidor para revisão
git push origin feature/TK-01-formulario-cadastro
```

1.3 GitHub e GitLab: onde o repositório vive

Git é a ferramenta local. GitHub e GitLab são plataformas web que hospedam o repositório na nuvem e adicionam funcionalidades de colaboração: revisão de código (Pull Requests), controle de acesso, integração com CI/CD e rastreamento de issues.

No projeto, o repositório está no GitHub. Qualquer pessoa do time pode clonar o projeto, criar uma branch, trabalhar e abrir um Pull Request para que o código seja revisado antes de entrar na branch principal.

Pull Request: a revisão de código

Um Pull Request (PR) é o mecanismo pelo qual uma branch pede para ser incorporada ao código principal. Antes do merge, ao menos um colega lê o código, faz comentários e aprova. Isso evita que erros cheguem a produção e distribui o conhecimento do sistema entre o time.

1.4 .gitignore e o que NÃO vai para o repositório

Nem tudo deve ser versionado. Senhas, chaves de API, arquivos gerados automaticamente e dependências do projeto nunca devem ir para o repositório. O arquivo .gitignore lista o que o Git deve ignorar.

```
# Exemplos do .gitignore do projeto

node_modules/      # dependências (baixadas com npm install)
.env               # variáveis de ambiente (senhas, chaves de API)
.env.local         # configurações locais do desenvolvedor
dist/              # arquivos gerados pelo build
*.log              # logs de execução
```

Métodos Ágeis

2.1 Por que métodos ágeis?

Durante décadas a indústria de software tentou planejar projetos inteiros antes de escrever a primeira linha de código - detalhando requisitos, arquitetura, cronogramas e orçamentos em documentos exaustivos. O resultado, na maioria dos casos, era um software que levava anos para ficar pronto e não mais refletia o que o usuário queria no momento da entrega.

Os métodos ágeis surgiram como resposta a esse problema. Em vez de planejar tudo de uma vez, o desenvolvimento é dividido em ciclos curtos (geralmente de 1 a 4 semanas). Ao final de cada ciclo, algo funcionando é entregue e validado com o usuário. O plano é ajustado com base no que se aprendeu.

O Manifesto Ágil em quatro valores

Valor Ágil	Significado prático
Indivíduos e interações	Conversas diretas valem mais que processos rígidos e ferramentas sofisticadas.
Software funcionando	Entregar algo que funciona é mais útil que documentação extensa sem código rodando.

Colaboração com o cliente	O usuário deve ser parceiro do desenvolvimento, não o destinatário final.
Responder a mudanças	Se o usuário muda de ideia ou surge um problema novo, o time se adapta em vez de seguir o plano cegamente.

2.2 Scrum: o framework mais usado

Scrum é o framework ágil mais adotado. Ele organiza o trabalho em torno de três papéis, três artefatos e cinco eventos. Vamos entender cada um com exemplos do nosso projeto.

Os três papéis do Scrum

Papel	Responsabilidade no projeto
Product Owner (PO)	Define o que o sistema precisa fazer e em que ordem. No nosso caso, e quem mais conversa com o cliente e traduz as necessidades em tarefas.
Scrum Master	Garante que o time siga o processo e remove impedimentos. Exemplo: se alguém está travado por falta de acesso ao servidor, o Scrum Master resolve.
Time de Desenvolvimento	As pessoas que constroem o sistema: designers, programadores front e back. No CodeLab, os próprios estudantes.

Os artefatos do Scrum

Artefato	No projeto
Product Backlog	A lista completa de tudo que o sistema precisa ter.
Sprint Backlog	A fatia do Product Backlog escolhida para o ciclo atual. Exemplo: na Sprint 1, o time pegou ST-01 (Cadastro de Acao) e TK-07 (Design System).
Incremento	O software funcionando ao final da Sprint. Não é um protótipo ou um rascunho: é o código que roda de verdade.

Os eventos do Scrum

1. Sprint Planning: reunião no início do ciclo para decidir quais tarefas o time vai pegar.
2. Daily Standup: reunião diária de 15 minutos onde cada pessoa diz o que fez, o que vai fazer e se tem algum impedimento.
3. Sprint Review: ao final do ciclo, o time mostra o que foi feito para o Product Owner (e, se possível, para o cliente).
4. Sprint Retrospective: o time reflete sobre o processo - o que funcionou, o que deve mudar.

2.3 O Backlog em detalhe: Épicos, Stories e Tasks

No arquivo de tarefas do projeto, você encontra três níveis de detalhamento. Entender essa hierarquia é fundamental para saber em qual escala cada pessoa está trabalhando.

Épicos (EP)

Um Épico é um objetivo de negócio amplo. Ele não é implementado diretamente, mas serve de agrupador para Stories e Tasks. No projeto, temos cinco Épicos:

- EP-01: Gestão de Ações Comunitárias - tudo que envolve criar, visualizar e participar de eventos.
- EP-02: Experiência do Usuário e Acessibilidade - garantir que o app funcione para pessoas com baixa familiaridade digital.
- EP-03: Funcionalidade Offline e Performance - o app funciona sem internet e consome pouco dado.
- EP-04: Notificações e Comunicados - avisar os participantes sobre eventos e mudanças.
- EP-05: Infraestrutura, Arquitetura e Segurança - a base técnica que sustenta tudo.

Stories (ST)

Uma Story (história de usuário) descreve uma necessidade do ponto de vista de quem usa o sistema. O formato clássico é: 'Como [persona], quero [acao], para [beneficio].'

Tasks (TK) e Sub-tasks (ST-K)

Uma Task é a unidade de trabalho técnico que o desenvolvedor realmente executa. Ela nasce de uma Story e já é específica o suficiente para ser estimada e desenvolvida por uma pessoa em poucos dias.

Task	O que faz
TK-01	Criar formulario de cadastro no frontend (React, mobile-first, acessível)
TK-02	Implementar endpoint POST /api/acoes no backend (validação Zod, PostgreSQL, rate limiting)
TK-07	Criar o design system mobile-first (cores, fontes, componentes reutilizáveis)
TK-18	Implementar autenticação OTP via WhatsApp
TK-17	Configurar pipeline de CI/CD

2.4 Kanban: visualizando o fluxo de trabalho

Kanban é um método visual de gestão que representa o estado de cada tarefa em um quadro com colunas. É mais simples que o Scrum e pode ser usado junto com ele ou de forma independente.

No Jira (ferramenta usada pelo projeto), cada tarefa passa pelas colunas: A Fazer, Em Progresso, Em Revisão e Concluído. O quadro Kanban deixa visível, em tempo real, onde cada tarefa está e se há gargalos.

QUADRO KANBAN - SPRINT 1 (exemplo ilustrativo)

A FAZER	EM PROGRESSO	EM REVISAO	CONCLUIDO
TK-04 (filtros)	TK-02 (API)	TK-01 (form.)	TK-07 (design)
TK-15 (notific.)	TK-08 (pesquisa)	TK-09 (telas)	
TK-18 (auth OTP)			

WIP Limit: evitar sobrecarga

Uma prática importante do Kanban é o limite de trabalho em progresso (Work In Progress Limit). Se cada pessoa só pode ter no máximo dois itens 'Em Progresso' ao mesmo tempo, o time é forçado a terminar o que começou antes de pegar algo novo. Isso evita a ilusão de que 'mais tarefas abertas = mais trabalho feito'.

2.5 Estimativas: Planning Poker e Story Points

Como o time sabe quanto trabalho cabe em uma Sprint? Usando estimativas relativas chamadas Story Points. Em vez de estimar em horas (o que tende a ser impreciso), o time classifica o esforço de cada tarefa em uma escala relativa - geralmente a sequência de Fibonacci: 1, 2, 3, 5, 8, 13, 21.

No Planning Poker, cada pessoa do time escolhe secretamente um número para a tarefa. Todos revelam ao mesmo tempo. Se há discordância grande (por exemplo, alguém deu 3 e outro deu 13), discute-se o motivo - e ali surgem os riscos e dúvidas escondidos na tarefa.

Arquitetura de Software

3.1 O que é arquitetura de software?

Arquitetura de software é o conjunto de decisões estruturais que definem como um sistema é organizado - quais são suas partes, como elas se comunicam e quais responsabilidades cada uma tem. Essas decisões são difíceis de mudar depois, por isso são tomadas cedo no projeto.

Uma boa analogia é a planta baixa de uma casa: você decide onde ficam os cômodos, as paredes e as instalações elétrica e hidráulica antes de começar a construir.

Mudar a localização de um banheiro depois das paredes prontas é custoso e trabalhoso. O mesmo vale para a arquitetura de um sistema.

3.2 Separação de responsabilidades: o princípio central

O princípio mais importante da arquitetura moderna é a separação de responsabilidades (Separation of Concerns). Cada parte do sistema deve ter uma função bem definida e não deve tentar fazer mais do que isso.

No projeto, isso se reflete na divisão entre frontend e backend:

- Frontend: responsável pela interface que o usuário vê e com a qual interage no celular.
- Backend: responsável pelas regras de negócio, persistência dos dados e segurança.
- Banco de dados: responsável por armazenar e consultar dados de forma eficiente.

Cada um desses níveis pode ser desenvolvido, testado e implantado de forma relativamente independente.

3.3 Padrões arquiteturais: MVC e Arquitetura em Camadas

Existem vários padrões estabelecidos para organizar o código de um sistema. Dois dos mais comuns são o MVC e a Arquitetura em Camadas. É importante conhecê-los porque você vai encontrar referências a eles em documentações, tutoriais e no dia a dia da profissão.

MVC (Model-View-Controller)

MVC é o padrão arquitetural mais ensinado em cursos de engenharia de software e muito comum em frameworks web. Ele divide a aplicação em três camadas com responsabilidades bem distintas:

Camada	Responsabilidade
Model (Modelo)	Representa os dados e as regras de negócio: as tabelas do banco de dados e as funções que as manipulam.
View (Visão)	O que o usuário vê: telas, formulários, listas. Em aplicações web, é o HTML/CSS renderizado.
Controller (Controlador)	O intermediário. Recebe uma requisição, decide o que fazer, chama o Model e devolve a resposta para a View.

O MVC faz muito sentido em sistemas onde o servidor monta e devolve páginas HTML prontas (o chamado Server-Side Rendering). Nesse fluxo, o usuário pede uma página, o Controller busca os dados no Model, e a View e o HTML gerado e enviado de volta.

Arquitetura em Camadas

A Arquitetura em Camadas organiza o sistema em níveis horizontais, onde cada camada tem uma responsabilidade específica e só se comunica com a camada imediatamente abaixo. É o padrão que estrutura o backend do projeto:

Camada	Responsabilidade no projeto
Rotas HTTP	Definem as URLs aceitas e os verbos HTTP (GET, POST, PUT, DELETE). Ponto de entrada de cada requisição.
Controllers	Recebem a requisição, validam as entradas com Zod e decidem qual serviço chamar.
Services	Contém a lógica de negócio: verificar se o usuário tem permissão, calcular dados, orquestrar operações.
Repositories	Traduzem chamadas de negócio em queries SQL. Única camada que fala diretamente com o PostgreSQL.
Banco de dados	PostgreSQL. Armazena e consulta os dados de forma persistente.

Essa separação traz um benefício prático imediato: se o time precisar trocar o banco de dados de PostgreSQL para outro, só a camada de Repositórios precisa mudar. O restante do código não é tocado.

DevOps

4.1 O que é DevOps?

DevOps é a cultura e o conjunto de práticas que aproximam as equipes de Desenvolvimento (Dev) e Operações (Ops). Antes do DevOps, o time que desenvolvia o software e o time que implantava e mantinha o servidor eram grupos separados, com objetivos frequentemente conflitantes: devs queriam lançar rápido, ops queriam estabilidade.

DevOps quebra essa barreira com automação, comunicação e responsabilidade compartilhada. O objetivo é que o software de qualidade chegue ao usuário de forma rápida, confiável e repetível.

4.2 CI/CD: o coração do DevOps

CI/CD (Continuous Integration / Continuous Delivery) é a prática de automatizar a validação e a entrega de cada mudança de código.

Integração Contínua (CI)

Cada vez que um desenvolvedor faz push de código para o repositório, uma série de verificações automáticas é executada:

1. o código segue as convenções de estilo definidas pelo time?
2. cada função faz o que se espera dela?
3. os endpoints da API respondem corretamente?
4. o projeto compila sem erros?

Se qualquer uma dessas etapas falhar, o desenvolvedor recebe uma notificação imediata e o código não é mesclado. Isso evita que bugs cheguem ao ambiente de produção.

Entrega Contínua (CD)

Depois que a CI aprova o código, a CD cuida de levar as mudanças ao servidor automaticamente. Em geral, o fluxo é:

```
1. Developer faz push para branch feature/TK-XX
  |
  v
2. GitHub Actions dispara automaticamente:
  -> Instala dependências (npm install)
  -> Roda lint (eslint)
  -> Roda testes (jest)
  -> Faz o build do frontend
  |
  v
3. Se tudo passou: Pull Request pode ser aprovado
  |
  v
4. Merge para main: deploy automático para ambiente de staging
  |
  v
5. Aprovação manual: deploy para produção
```

4.3 Ambientes: desenvolvimento, staging e produção

Um projeto profissional sempre tem pelo menos três ambientes separados, cada um com um propósito específico:

Ambiente	Propósito e características
Desenvolvimento (local)	Cada desenvolvedor roda o projeto no próprio computador. Pode ter dados falsos, logs verbosos e configurações específicas. Erros aqui só afetam o desenvolvedor.
Staging (homologação)	Cópia fiel da produção, mas com dados de teste. É onde o Product Owner e, idealmente, alguns usuários testam antes do lançamento. Deploy automático a cada merge na main.

Produção

O ambiente real. Mudanças só chegam aqui após aprovação. Alta disponibilidade e monitoramento ativo.

4.4 Variáveis de ambiente

Cada ambiente tem configurações diferentes: URL do banco de dados, chave da API do Firebase (para notificações push), segredo para assinar tokens de autenticação. Essas configurações nunca ficam no código - elas são injetadas via variáveis de ambiente no momento da execução.

```
# Exemplo de arquivo .env (NUNCA vai para o Git)

DATABASE_URL=postgresql://user:senha@localhost:5432/maes_db
JWT_SECRET=chave-secreta-muito-longa-e-aleatoria
FIREBASE_PROJECT_ID=maes-mobilizadoras
FIREBASE_SERVER_KEY=AAAA...
NODE_ENV=development
```

4.5 TDD: testes antes do código

Uma prática fortemente incentivada no projeto (presente nas definições de todas as Tasks do Jira) e o TDD - Test-Driven Development (Desenvolvimento Guiado por Testes). A ideia é escrever o teste antes de escrever o código que o satisfaz.

O ciclo do TDD tem tres passos:

1. Red: escreva um teste que descreve o comportamento desejado. Rode-o. Ele falha (o código ainda não existe).
2. Green: escreva o código mais simples possível para o teste passar.
3. Refactor: melhore o código sem quebrar o teste.

Como Tudo se Conecta

5.1 A jornada de uma feature do Jira até a produção

Agora que você conhece repositórios, métodos ágeis, arquitetura e DevOps, vamos ver como esses conceitos trabalham juntos em um cenário real do projeto.

Cenário: o time decidiu implementar a Story ST-01 (Cadastro de Ação Comunitária) nesta Sprint.

1. Sprint Planning: o time seleciona ST-01 e suas Tasks (TK-01, TK-02, ST-K-01) para a Sprint. Estimam: TK-01 = 5 pontos, TK-02 = 8 pontos, ST-K-01 = 3 pontos.
2. Git: cada desenvolvedor cria uma branch a partir da main (feature/TK-01-form, feature/TK-02-api).

3. Arquitetura: o desenvolvedor frontend implementa os componentes. O desenvolvedor backend implementa o endpoint POST /api/acoes (Controller na Arquitetura em Camadas) e o schema do banco (Repository + Model).
4. TDD: antes de codar, escrevem os testes definidos no Jira (POST válido, campos ausentes, data no passado).
5. Daily Standup: cada pessoa informa o progresso e pede ajuda se precisar.
6. Pull Request: ao terminar, cada desenvolvedor abre um PR. Um colega revisa o código e aprova.
7. CI: o GitHub Actions roda lint + testes + build automaticamente. Se passar, o PR pode ser mesclado.
8. Merge: o código entra na main. O CD faz deploy automático para staging.
9. Sprint Review: o Product Owner testa no staging. Se possível, uma usuária real também testa.
10. Produção: aprovado no staging, o deploy vai para produção.

Inteligência Artificial como Ferramenta de Desenvolvimento

6.1 Por que falar de IA neste guia?

A IA generativa mudou a rotina de desenvolvedores em todo o mundo nos últimos anos. Ferramentas como Claude, ChatGPT, GitHub Copilot e Cursor deixaram de ser curiosidades para se tornarem parte do fluxo de trabalho diário de times profissionais.

Para quem está começando, a IA pode ser especialmente útil: ela explica conceitos técnicos em linguagem acessível, gera código de exemplo, aponta erros e responde perguntas a qualquer hora. O objetivo deste capítulo é mostrar onde e como usar essas ferramentas de forma eficaz - e também onde tomar cuidado.

6.2 O que a IA faz bem no desenvolvimento de software

Explicar código desconhecido

Você encontrou um trecho de código que não entende - seja no próprio projeto, seja em um tutorial. Cole o código no chat e pergunte: 'O que esse código faz, passo a passo?' A IA quebra o código em partes e explica cada uma com linguagem simples.

Exemplo prático

Dúvida: 'O que significa esse trecho do schema Zod na TK-02?'

```
z.object({  
  titulo: z.string().min(3).max(120),  
  data: z.string().datetime().refine(d => new Date(d) > new Date()),  
})
```

Resposta da IA: 'Esse objeto valida que a requisição tem um campo título com entre 3 e 120 caracteres, e um campo data que seja uma string no formato ISO 8601 E que represente uma data futura em relação ao momento da requisição.'

Gerar código de partida (boilerplate)

Criar do zero a estrutura de um componente, um endpoint ou uma migration de banco de dados consome tempo e é propenso a erros de digitação. A IA gera esse esqueleto em segundos, e você ajusta para o contexto específico.

Depurar erros (debugging)

Recebeu uma mensagem de erro que não faz sentido? Cole o erro completo junto com o trecho de código relevante e pergunte: 'Por que esse erro acontece e como corrigir?'

A IA é especialmente boa com erros comuns de tipagem, erros de CORS, problemas de autenticação JWT e mensagens de erro do PostgreSQL - todos frequentes neste projeto.

Escrever testes

O projeto define testes para cada Task no Jira. A IA pode gerar o código dos testes a partir da descrição em texto dos cenários:

Exemplo pratico

Prompt: 'Escreva testes Jest para o endpoint POST /api/acoes. Cenários:

- 1) POST válido retorna 201 com o evento criado.
- 2) Body sem título retorna 400.
- 3) Data no passado retorna 400.'

A IA gera os três testes com describes, its e expects. Você verifica se os asserts batem com o comportamento real do endpoint.

Entender conceitos novos rapidamente

Neste capítulo falamos de Scrum, Git, REST, Arquitetura em Camadas, CI/CD e TDD em um nível introdutório. Qualquer um desses tópicos tem livros inteiros escritos sobre

ele. A IA funciona como um tutor disponível 24 horas: você pode perguntar 'Explica WIP Limit como se eu tivesse 15 anos' ou 'Qual a diferença entre PUT e PATCH na pratica?' e receber uma resposta adaptada ao seu nível.

Revisar Pull Requests

Antes de abrir um Pull Request para revisão humana, você pode pedir a IA que revise o código: 'Tem algo que pode ser melhorado nesse trecho? Há problemas de segurança ou performance? Isso não substitui a revisão do colega, mas ajuda a pegar problemas óbvios antes.

6.3 Como pedir melhor: a arte do prompt

A qualidade da resposta da IA depende diretamente da qualidade do que você pergunta. Um prompt vago gera uma resposta genérica. Um prompt específico e contextualizado gera uma resposta útil.

Elemento do prompt	Por que ajuda
Contexto do projeto	'Estou fazendo um projeto usando Python com Flask e PostgreSQL.' A IA não adivinha o ambiente.
O que você quer exatamente	'Gere o código' vs 'Explique o código' vs 'Aponte erros no código' são pedidos completamente diferentes.
O que você já tentou	Compartilhar o que já foi tentado evita respostas repetindo o que você já sabe.
Restrições ou formato	'Responda em no máximo 5 pontos' ou 'Use bibliotecas externas' direcionam a resposta.

Comparação: prompt ruim vs prompt bom

RUIM: 'Como faço um endpoint?'

BOM: Estou usando Python com Flask no backend de um sistema de gestão de eventos. Preciso de um endpoint GET /api/acoes que aceite query params de filtro (data_inicio, data_fim, status) e retorne uma lista paginada de eventos do PostgreSQL. Já tenho o modelo Event definido. Mostre apenas o controller e o service, sem o repository.'

O segundo prompt leva a IA diretamente ao ponto relevante para o seu problema.

6.4 Onde tomar cuidado

A IA é uma ferramenta poderosa, mas tem limitações reais que um desenvolvedor iniciante precisa conhecer para não cair em armadilhas.

A IA erra - e com confiança

Modelos de linguagem podem gerar código que parece correto mas tem bugs sutis, usar APIs que não existem mais, ou inventar comportamentos de bibliotecas. Nunca cole código gerado por IA em produção sem ler e entender cada linha.

Não substitui entender o problema

Se você não entender o que o código faz, não vai saber quando ele está errado. Use a IA para aprender e acelerar, não para pular o entendimento. Um desenvolvedor que depende de IA para tudo tem dificuldade de depurar quando algo dá errado e não tem IA por perto.

Cuidado com informações sensíveis

Nunca cole no chat de uma IA pública: senhas, chaves de API, tokens de acesso, dados pessoais de usuários ou qualquer informação confidencial. Use dados fictícios para ilustrar o problema.

Versões de bibliotecas e frameworks

A IA tem uma data de corte de conhecimento. Ela pode gerar código usando a API de uma versão antiga de uma biblioteca. Sempre confira na documentação oficial se o código gerado é compatível com a versão que o projeto usa.

Front-End

A estrutura de Frontend utilizada no projeto foi de uma divisão simples de arquivos de HTML, CSS e JavaScript puros, sem nenhum uso de Frameworks como Ionic ou Angular, visando criar um aplicativo mais simples que não seja afetado pela instabilidade de rede. Vários elementos visuais, como header, footer, calendário e blocos com informações dos eventos foram criados como componentes, ou seja, trechos de código prontos reutilizáveis em diversas telas ou partes da mesma tela.

Cada tela principal do aplicativo, como a tela de ações comunitárias, de calendário, avisos e perfil possuem um arquivo HTML próprio, chamando múltiplos scripts e stylesheets respectivos a cada componente utilizado. A única exceção para essa abordagem é a tela de cadastro, que não possui nenhuma componente e é formada por um único arquivo HTML dividido em segmentos através de JavaScript.

As seguintes divisões explicam o passo a passo e dicas de como utilizar a Inteligência Artificial para auxiliar programadores a desenvolverem telas de aplicativo/páginas web de simples de maneira eficiente.

HTML

A estrutura HTML de todas as telas do aplicativo foram desenvolvidas manualmente. Entretanto, a Inteligência Artificial atuou no planejamento inicial da arquitetura da aplicação como uma orientadora de boas práticas de desenvolvimento, auxiliando na definição da melhor e mais profissional organização de código.

Tela de Cadastro:

Inicialmente, pretendia-se criar arquivos separados para cada etapa de cadastro(telefone, código SMS e nome de usuário/bairro de origem), conectados pela tag <a>. Contudo, o agente sugeriu que todas as etapas do fluxo permanecessem em um único arquivo, evitando carregamentos desnecessários de múltiplas páginas e proporcionando uma navegação mais fluida e eficiente.

Telas principais:

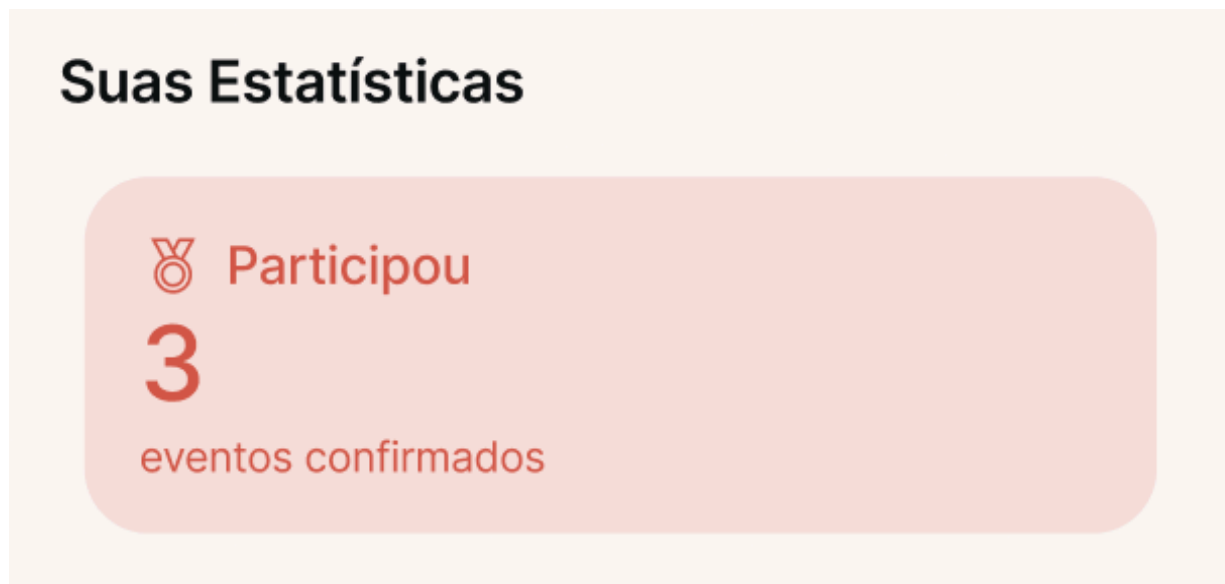
Novamente foi-se questionado em relação à abordagem correta em relação à estrutura HTML das páginas. Dessa vez foi sugerido que o desenvolvimento das principais telas do aplicativo(ações comunitárias, calendário, avisos, perfil de usuário) ocorresse de forma separada, em contraste com a tela de cadastro, sob a justificativa de que as telas servem propósitos distintos e não seguem um caminho contínuo de ações.

Como usar IA para HTML:

Seja bem específico em relação a estrutura visual: Se o objetivo é criar um arquivo HTML inteiro apenas com IA, não basta apenas mencionar para o modelo qual tipo de página ou tela é desejado. Além de listar quais elementos e funcionalidades devem estar presentes na tela, é essencial também explicitar quais são os agrupamentos desses elementos e as posições relativas entre eles dentro deste espaço. Esse detalhamento

permite que o modelo crie as divisões <div> apropriadas envolvendo as peças corretas, facilitando assim o desenvolvimento do arquivo .css.

Por exemplo, para reproduzir um resultado similar ao da imagem, deve-se dizer ao modelo que esses três textos pertencem ao mesmo compartimento, o que permite o modelo a entender que se trata de um caso de inclusão de div. Essa prática serve também para elementos posicionados de forma não automática, como a imagem alinhada com o primeiro texto. Um grande nível de detalhamento é recomendado.



Exemplo de prompt: “quero apenas o código em HTML de um componente simples para um aplicativo. Aqui estão os elementos que eu quero: os textos "Participou", "3", "eventos confirmados" dentro de um mesmo compartimento, nessa ordem e um em cima do outro. Quero o primeiro texto a minha imagem "medalha.svg" na mesma linha.”

Resposta: <div class="componente-eventos">
 <div class="titulo">

 Participou
 </div>

 3

 eventos confirmados
</div>

Observe que há duas divisões: uma englobando todos os elementos, e outra encapsulando a imagem e o primeiro texto, o que permite um alinhamento horizontal entre eles através de CSS.

Use a Inteligência Artificial para planejamento e orientação sobre boas práticas: Como feito durante o projeto, o modelo de IA pode servir como um instrutor, dizendo quais medidas são as mais apropriadas para cada caso e orientando o

programador a formatar e estruturar o código de maneira mais profissional, legível e sustentável.

Exemplo de prompt: “Estou desenvolvendo a parte do cadastro de usuário para o aplicativo, dividido nas seguintes etapas: telefone, validação por código SMS, e Nome + Bairro. Qual abordagem você acha que é mais profissional e sustentável: criar arquivos .HTML para cada passo ou um único arquivo segmentado com <div>?”

Resposta(resumida): A abordagem mais profissional e sustentável para esse caso é usar um único arquivo HTML segmentado com <div>, exibindo uma etapa por vez via JavaScript.

Isso reduz:

- carregamentos desnecessários;
- repetição de estrutura/layout;
- quantidade de arquivos;
- complexidade de navegação entre etapas.

Como todas as telas pertencem ao mesmo fluxo de cadastro, essa abordagem costuma ser mais limpa e eficiente em aplicações frontend simples sem framework.



É importante ressaltar que muitas vezes o agente insere tags <div> e outros elementos em locais inadequados, além de inserir componentes extras, como o <style> com estilização pronta no arquivo HTML, logo o código gerado deve ser sempre verificado para evitar futuros problemas de estilização.

CSS

Todos os arquivos .css do projeto foram criados manualmente. Em raras ocasiões houveram conflitos de estilização (por exemplo, um atributo padding pode sobrescrever uma declaração anterior.) causando inconsistências nos posicionamentos ou cores dos elementos visuais. Assim, a Inteligência Artificial se responsabilizou pela localização exata das linhas de código problemáticas.

Como usar IA para CSS:

Defina os nomes das classes e IDs dos elementos: As tags do arquivo HTML devem estar marcadas com classes e/ou IDs com nomes informativos, para posterior estilização individual em CSS. É possível pedir para que o modelo gere automaticamente o código com os rótulos já prontos.

Exemplo de prompt: “Agora preciso estilizar esse componente com um arquivo .css. Insira nas tags classes e IDs com nomes intuitivos e informativos.”

Resposta(resumida):

```
<div id="eventos-confirmados-card" class="card-eventos-confirmados">

  <div class="card-eventos-topo">
    

    <span class="texto-participou">
      Participou
    </span>
  </div>

  <span class="numero-eventos-confirmados">
    3
  </span>

  <span class="texto-eventos-confirmados">
    eventos confirmados
  </span>

</div>
```

Criação de um arquivo .css: Assim como em HTML, é imprescindível um alto nível de detalhamento ao fazer o prompt. É altamente recomendado mencionar o espaçamento e posicionamento de cada elemento pertencente a um mesmo compartimento, se estão dispostos em linhas, colunas, etc. Por fim, para um resultado mais profissional, deve-se buscar a responsividade e tornar explícito a organização desejada das partes para cada tamanho de tela (mobile, tablet e desktop).

Exemplo de prompt: Agora quero o código .css para este componente. Quero que todos os elementos estejam dentro de uma "caixa" vermelha, de cor #F8DDDA, com bordas arredondadas. Os três textos também devem ter a cor vermelha #D35746 e um espaçamento pequeno entre eles. A imagem e o primeiro texto devem estar alinhados horizontalmente, um do lado do outro. Por fim, não quero que todas essas partes estejam "grudadas" na caixa, quero um espaçamento interno.

Resposta:

```
#eventos-confirmados-card {
  background-color: #F8DDDA;
  border-radius: 16px;
  padding: 16px;

  display: flex;
  flex-direction: column;
  gap: 6px;

  width: fit-content;
}

.card-eventos-topo {
  display: flex;
  align-items: center;
  gap: 6px;
}

.texto-participou,
.numero-eventos-confirmados,
.texto-eventos-confirmados {
  color: #D35746;
}

.icone-medalha {
  width: 20px;
  height: 20px;
}
```

Use a IA para identificar erros e inconsistências: A sobreposição de declarações e pequenos erros são consideravelmente difíceis de serem localizados, principalmente em arquivos .css grandes e com muitos elementos. Assim, a IA torna-se útil e otimizadora ao fazer essa tarefa em poucos segundos, dizendo ao programador a linha de código exata responsável pelo problema.

JavaScript

O JavaScript foi responsável por gerenciar a interatividade, o estado da aplicação, a comunicação assíncrona com o back-end e a manipulação dinâmica de componentes. Dado o objetivo de criar um aplicativo leve e resiliente a conexões instáveis, priorizou-se o uso de vanilla JavaScript, com ênfase em código modular, eficiente e de fácil manutenção.

A Inteligência Artificial foi amplamente utilizada tanto na implementação de funcionalidades quanto na revisão contínua da arquitetura, contribuindo para evitar a acumulação de código “spaghetti” comum em aplicações front-end simples.

Como usar IA para JavaScript: Fluxo Recomendado

1. **Definição clara do comportamento** Descreva detalhadamente o fluxo completo da funcionalidade antes de solicitar código. Inclua elementos envolvidos, estados da interface, integrações com API e tratamentos de erro.
2. **Solicitação do código com restrições** Informe explicitamente: “usar apenas vanilla JavaScript”, “sem frameworks”, “considerar conectividade instável” e “seguir a estrutura de arquivos já definida”.
3. **Revisão arquitetural** Após receber o código, utilize a IA para validar responsabilidades, sugerir refatorações e verificar aderência ao princípio da responsabilidade única.
4. **Integração e refatoração** Teste a funcionalidade no navegador, identifique problemas e retorne à IA para correções iterativas.

Exemplo de prompt eficaz

“Desenvolva o arquivo `js/components/avisos.js` para o aplicativo MaMobi. Deve conter:

- Função assíncrona `carregarAvisos()` que realiza GET em `/api/avisos`, trata loading e erros de rede, e renderiza cards na div `#lista-avisos` usando template literals.
- Função `criarAviso()` que coleta dados do modal, envia POST e atualiza a lista em caso de sucesso.
- Bloqueio de rolagem durante o modal e feedback visual adequado. Mantenha as funções modulares e adicione comentários explicativos.”

Cuidados importantes

- Especifique à IA o nível de complexidade desejado para evitar soluções excessivamente elaboradas.
- Documente as principais funções para facilitar a compreensão futura.

Back-End

A presente seção orienta como utilizar assistentes de IA de forma estratégica para acelerar o desenvolvimento de backends mantendo a integridade arquitetural e a confiabilidade do sistema. Foi utilizada a linguagem Python com as bibliotecas Flask, SQLAlchemy e Pydantic.

Passo a passo

1. Escolha seus aliados

Há muitas maneiras em que você pode usar IA no projeto. A mais óbvia é abrir um chatbot, como o ChatGPT ou Deepseek R1, e copiar e colar código do chat para seu projeto e do seu projeto para o chat. Essa abordagem tem a vantagem de aumentar o controle do humano sobre o processo, restringindo o que a IA pode fazer. Porém, na prática, isso pode ficar ineficiente e repetitivo se você estiver usando bastante a IA, e é aí que entram os agentes.

Agentes são basicamente chatbots com a capacidade de interagir com o computador em que está sendo executado de maneira direta. Eles podem criar, editar e excluir arquivos e diretórios, executar comandos, entre outros. Alguns, como o Claude Code e o Gemini CLI, estão atrelados a modelos de empresas específicas (respectivamente Anthropic e Google), enquanto outros, como o aider, dão maior flexibilidade, permitindo até mesmo a execução de modelos locais. No entanto, a área da inteligência artificial artificial avança muito rápido, e esse parágrafo inteiro provavelmente já está desatualizado. Faça sua pesquisa e chegue a suas conclusões.

2. Saber exatamente o que quer fazer

Se você não sabe exatamente o escopo do seu projeto, como vai explicar isso para a inteligência artificial? Os agentes precisam de instruções específicas para se comportarem de maneira previsível. Então, planeje antes de construir.

Antes de tudo, defina e entenda o público alvo. Depois, usando sua filosofia de design favorita (por exemplo, Design Thinking), defina quais serão os requisitos funcionais (ou seja, quais funcionalidades o sistema terá, qual será seu comportamento) e os não funcionais (isto é, qual desempenho precisa ter, quantas requisições por segundo deve suportar, etc.).

Sabendo os requisitos, pense na arquitetura do sistema. Será monolito ou em microsserviços? Qual banco de dados? Quais ferramentas e linguagens de programação? Quais serão os módulos? Quais padrões de projeto? Se ficar em dúvida, peça ajuda à IA para tomar essas decisões. Porém, a não ser que isso vá contra os requisitos do projeto, é recomendável adotar a linguagem Python com a biblioteca Flask, pois são as tecnologias predominantes nos conjuntos de treinamento dos agentes de IA (<https://arxiv.org/pdf/2503.17181v1>).

3. Esqueleto do projeto

Antes de começar a desenvolver, é importante definir a estrutura dos diretórios do projeto. Isso dependerá das ferramentas escolhidas na etapa anterior. Porém, algumas recomendações gerais são:

- Separar frontend e backend;
- Separar código-fonte funcional, documentação e testes automatizados.
Um exemplo de possível organização é:
- backend/
 - src/
 - main.py
 - test/
 - ...
 - doc/
 - ...
- frontend/
 - ...

Outra coisa importante é configurar versionamento. Durante o desenvolvimento com IA, provavelmente vários erros serão cometidos, e será necessário restaurar versões anteriores do projeto ou de arquivos individuais com versatilidade. A ferramenta mais comum, e portanto mais presente no conjunto de treinamento das IAs, é o git. Embora saber usar git seja bem útil, agentes são capazes de executar comandos git diretamente, facilitando o versionamento caso não tenha tanta experiência.

4. Ciclo de construção e aprimoramento

Chegou a hora de começar a implementar. Primeiramente, separe as informações relevantes levantadas na etapa 2 acessíveis à IA. Isso pode ser feito criando um documento .txt com essas informações ou incluindo elas diretamente nos prompts. Se usar um arquivo de texto, lembre-se de mencioná-lo ao agente, para que ele seja lido.

Agora, peça à IA para implementar uma parte pequena do projeto, por exemplo, uma parte de um módulo ou um endpoint da API. Inclua no prompt um pedido para “escrever testes automatizados” para que a validade da lógica possa ser verificada. Verifique rapidamente o código para garantir que não tenha malware e execute os testes.

Depois, leia os códigos escritos. Escreva a documentação do código, adicionando comentários e outros documentos que expliquem o funcionamento do sistema. Além de facilitar que outras pessoas entendam o sistema, o fato de você escrever a documentação garante que você entendeu o código produzido. Isso também é importante pois, às vezes, a implementação da IA está tecnicamente correta e, portanto, todos os testes passarão, embora o que a IA fez não se alinhe aos objetivos do projeto. Por exemplo, se você pedir uma função que calcule a raiz quadrada de um dado número, a IA pode fazer uma que dispare um erro em caso de entrada negativa e escrever testes que garantam tal comportamento, em contraste com o que você queria da função, que seria retornar um número complexo. Além disso, sem esse entendimento, a base de código pode atingir um nível de complexidade tão alto que entendê-la se torna muito difícil e, nesse caso, até a inteligência artificial pode se perder nele.

Então, leia os testes para garantir que cobrem casos de borda e que eles, de fato, estejam validando a lógica do código. Mesmo se sentir que os testes estão suficientes, peça para a IA pensar em mais testes e casos de borda. Novamente, verifique se há malware e execute os testes. Agentes espertos executarão os testes sem que você peça e corrigirão o código com base nos casos que falharam.

Finalmente, faça um ou mais commits com a ferramenta de versionamento que escolheu. As melhores mensagens de commit têm títulos sucintos mas descritivos, com um

corpo bem detalhista. Quando estiver pronto, repita o ciclo novamente até a completude do projeto.

Recomendações chave

Defina Restrições de Design Imediatamente

IA tendem a usar padrões genéricos que podem não servir ao seu projeto. Lembre-se: os dados de treinamento das IAs incluem desde os exemplos do livro "Clean Code" até o blog do estudante que acabou de aprender "Hello, World".

- O que fazer: Antes de pedir o código, especifique padrões como "Prefira composição sobre herança" ou "Use apenas tipos compatíveis com SQLite". Naturalmente, isso exige um planejamento prévio da arquitetura geral do sistema.
- Por que: Isso evita refatorações custosas após o código já ter sido gerado.

Forneça Contexto Estruturado

Não peça para a IA "adivinhar" o banco de dados nem os objetivos do projeto. Escreva documentos em arquivos de texto como "Explicação de Tabelas.txt", "Dicionário de Dados.txt", "Escopo do Projeto.txt". Peça explicitamente para a IA ler esses arquivos.

- Exemplo: "Baseado nas definições em `docs/schema.txt`, gere os modelos Flask-SQLAlchemy."
- Resultado: O sistema produzido será mais coeso/consistente e mais alinhado às expectativas dos desenvolvedores.

Delegue a Lógica, mas Exija a Validação

É difícil ter certeza que a IA acertou a lógica do código. Você pode revisar o código e verificar isso, mas isso só move o problema: é difícil ter certeza que sua revisão foi completa. Isso é ainda mais preocupante se o código tiver uma lógica complexa. Portanto, sempre peça a implementação acompanhada de testes unitários ("pytest"). Se a lógica for crítica, use o comando: "Ainda não estou convencido da robustez, escreva mais 3 casos de teste cobrindo [cenário X, Y e Z]".

- O que observar: Se os testes passarem de primeira, a lógica é provavelmente sólida. Se falharem, a IA terá um rastro para se autocorrigir. Depois, escreva/peça mais testes, até que esteja convencido da confiabilidade do sistema.
- Resultado: Código mais confiável.

Force o uso de Idiomatismos da Biblioteca

Evite que a IA escreva funções manuais para tarefas que a biblioteca já resolve. Por exemplo, se pedir para a inteligência artificial implementar um sistema de login, ela pode começar a escrever lógica complexa de criptografia e autenticação. Isso pode levar à dificuldade de manutenção do código e à introdução de bugs em casos de borda. Então, se perceber que a IA está escrevendo algo que você acredita que alguém já tenha feito em uma biblioteca, pergunte a si mesmo e talvez ao agente: "Será que não existe um método nativo para isso?"

No desenvolvimento do presente projeto, esse fenômeno foi notado quando Gemini escreveu uma função para converter um modelo Pydantic para JSON listando campo por

campo. A autora humana, supondo que esse caso de uso é comum demais para não já ter sido implementado, interrompeu o agente e o perguntou: "No Pydantic não tem um método nativo para converter modelos para JSON?"

- Resultado: Código mais limpo, fácil de manter e que aproveita otimizações da linguagem.